합성함수 미분법에서 Backpropagation까지

Chain Rule → Gradient/Jacobian → Backpropagation in Linear Regression

이번 주의 핵심 질문

딥러닝에서 학습한다는 것은 결국 손실함수 L을 모델 파라미터에 대해 미분하고, 그 미분값을 이용하여 파라미터를 수정하는 과정이다. 그렇다면 수많은 함수가 반복적으로 합성된 신경망에서

$$\frac{\partial L}{\partial \theta}$$

를 어떻게 체계적으로 계산할 수 있을까?
이번 주에는 그 출발점이 되는 **chain rule**을 일변수에서 다변수로 확장하고, 가장 단순한 학습모델인 선형회귀를 계산 그래프로 다시 해석하여 **forward propagation과 backward propagation의 원형**을 직접 유도하고 구현한다.

학습목표

이 주차가 끝나면 다음을 설명하고 직접 계산할 수 있어야 한다.

1. 합성함수와 chain rule을 "변화율의 전달"이라는 관점에서 설명한다.
2. 편미분, gradient, Jacobian의 의미를 구분한다.
3. 다변수 chain rule을 scalar form과 matrix form으로 표현한다.
4. 계산 그래프(computational graph)에서 forward와 backward의 의미를 설명한다.
5. 선형회귀의 MSE loss에 대해 $\partial L/\partial w, \partial L/\partial b$를 chain rule로 유도한다.
6. 위 유도식을 NumPy로 직접 coding하여 gradient descent를 수행한다.

1 합성함수와 일변수 Chain Rule

1.1 합성함수와 chain rule

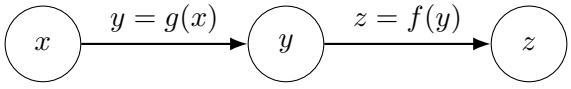
변수 x, y, z가 다음과 같이 연결되어 있다고 하자.

$$y = g(x), \qquad z = f(y).$$

그러면 z는 결국 x의 함수이고,

$$z = f(g(x)) = (f \circ g)(x)$$

로 쓸 수 있다.



여기에서 y는 단순한 기호가 아니라, x의 변화가 z로 전달되는 과정에 존재하는 **중간 변수(intermediate variable)**이다. 이 관점이 뒤에서 computational graph를 이해할 때 중요하다.

g가 x에서 미분가능하고 f가 $g(x)$에서 미분가능하면

$$h(x) = f(g(x))$$

의 도함수는

$$\boxed{h'(x) = f'(g(x))g'(x)}$$

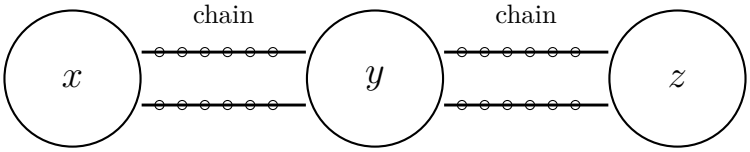
이다.

중간 변수 $y = g(x)$를 사용하면

$$\boxed{\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}}$$

로 쓸 수 있다.

연쇄법칙은 영어로 **chain rule**이라 하는데 이는 다음과 같이 이해할 수 있다.



크기가 다른 세 개의 톱니바퀴 x, y, z가 있다고 하자. x는 y에 체인으로 연결되어 있고, y는 z에 체인으로 연결되어 있다고 하자. x를 돌리면 y가 돌아가고 y에 연결된 z가 돌아가게 되므로, 중간 과정 y를 생략하고 생각하면 톱니바퀴 z의 변화는 x에 의해 결정된다.

만약 x, y, z의 톱니바퀴 수가, x가 한 바퀴 돌아가면 y가 두 바퀴 돌아가고, y가 한 바퀴 돌아갈 때 z는 세 바퀴 돌아가는 비율로 정해져 있다면, x가 두 바퀴 돌면 z는 몇 바퀴 돌아가겠는가?

이를 수학적으로 생각하면 다음과 같다.

$$x \text{ 변화} : y \text{ 변화} = 1 : 2 \quad \Longleftrightarrow \quad \frac{dy}{dx} = 2,$$

$$y \text{ 변화} : z \text{ 변화} = 1 : 3 \quad \Longleftrightarrow \quad \frac{dz}{dy} = 3.$$

따라서,

$$x \text{ 변화} : z \text{ 변화} = 1 : 6$$

이고,

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx} = 3 \times 2 = 6.$$

그러므로,

$$\begin{aligned}\Delta z &= (z\text{의 } y\text{에 대한 변화율}) \times \Delta y \\ &= \frac{dz}{dy}\Delta y \\ &= \frac{dz}{dy}\,(y\text{의 } x\text{에 대한 변화율})\,\Delta x \\ &= \frac{dz}{dy}\frac{dy}{dx}\Delta x.\end{aligned}$$

로 생각할 수 있고, 따라서 x가 두 바퀴 돌면 z는 12바퀴가 돌아갈 것이다.

한편 z를 x의 함수로 직접 생각하면

$$\Delta z \approx \frac{dz}{dx}\Delta x$$

이므로, 위의 식과 비교하여

$$\boxed{\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}}$$

인 것을 알 수 있다.

이것이 합성함수의 미분법, 즉 미분계수(변화율)

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

의 의미이다. 이 비유의 핵심은 중간 변수 y를 거치면서 **변화율이 연쇄적으로 전달된다**는 데 있다.

또한 derivative는 단순한 기호 조작이 아니라

$$\boxed{\text{입력의 작은 변화}(\Delta x)\text{가 출력의 변화}(\Delta z)\text{로 얼마나 증폭 또는 축소되는가}}$$

를 나타내는 **sensitivity**이다.

1.2 Local linearization으로 보는 미분

미분가능한 함수 f는 x 근처에서

$$f(x+\Delta x) = f(x) + f'(x)\Delta x + o(|\Delta x|)$$

로 근사된다. 즉,

$$\boxed{\Delta f \approx f'(x)\Delta x}$$

이다.

두 함수 $y = g(x)$, $z = f(y)$를 각각 선형근사하면

$$\Delta y \approx g'(x)\Delta x, \qquad \Delta z \approx f'(y)\Delta y,$$

따라서

$$\Delta z \approx f'(g(x))g'(x)\Delta x.$$

이것이 chain rule의 가장 중요한 구조이다.

1.3 예제

예제 1.

$$y = (x^3 + 1)^2.$$

$u = x^3 + 1$ 이라 두면 $y = u^2$ 이므로

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx} = 2u \cdot 3x^2 = 6x^2(x^3 + 1).$$

예제 2.

$$y = \sin(2x + 1).$$

$u = 2x + 1$ 이라 두면

$$\frac{dy}{dx} = \cos u \cdot 2 = 2\cos(2x + 1).$$

예제 3. 세 단계 합성

$$y = \exp\big(\sin(x^2)\big)\,.$$

중간 변수를

$$u_1 = x^2, \qquad u_2 = \sin u_1, \qquad y = e^{u_2}$$

로 두면

$$\frac{dy}{dx} = \frac{dy}{du_2}\frac{du_2}{du_1}\frac{du_1}{dx} = e^{u_2}\cos u_1\,(2x).$$

따라서

$$\boxed{\frac{dy}{dx} = 2x\cos(x^2)e^{\sin(x^2)}}.$$

2 다변수함수의 연쇄법칙

2.1 하나의 입력변수에서 여러 중간변수를 거치는 경우

다변수함수

$$z = f(x_1, x_2, \ldots, x_n)$$

가 각 변수 x_i 에 대하여 편미분 가능하다고 하자.

그런데 (알고보니) 각 변수 x_i 가 다시 하나의 변수 t 에 의하여 정해지는 함수

$$x_i = x_i(t), \qquad i = 1, \ldots, n$$

라고 하자. (x_i 를 변수를 나타내는 기호와 함수 $x_i(\cdot)$ 를 나타내는 기호로 동시에 사용했다.)

그러면 결국 z는

$$z = f\big(x_1(t), x_2(t), \ldots, x_n(t)\big)$$

와 같이 t에 대한 함수가 된다. 이 관계를 graph로 나타내면 다음과 같다.

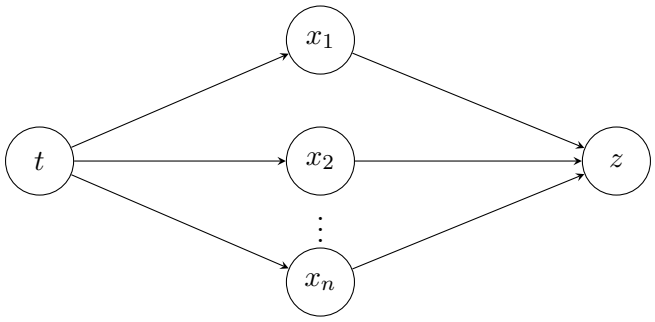


Figure 1: t의 변화가 여러 중간변수 x_i를 통하여 z에 전달되는 구조

이 graph를 따라 생각하면, t가 변할 때 $x_1, x_2, \ldots, x_n$이 각각 변하고, 각 x_i의 변화가 다시 z의 변화에 영향을 준다.

우리는 z의 변화는 x_i의 변화에 의한 영향의 총합으로 결정된다는 것을 알고 있으므로, z의 작은 변화 Δz는 일차근사식

$$\Delta z \approx \frac{\partial z}{\partial x_1}\Delta x_1 + \frac{\partial z}{\partial x_2}\Delta x_2 + \cdots + \frac{\partial z}{\partial x_n}\Delta x_n$$

으로 표현된다. 이것을 미적분학에서는 전미분

$$dz = \frac{\partial z}{\partial x_1}dx_1 + \frac{\partial z}{\partial x_2}dx_2 + \cdots + \frac{\partial z}{\partial x_n}dx_n$$

이라 한다.

그런데 각 x_i가 t의 함수이므로 x_i의 변화는 다시 t의 변화에 의하여 결정된다. 즉, t가 Δt만큼 변하면

$$\Delta x_i \approx \frac{dx_i}{dt}\Delta t$$

이다. 따라서

$$\begin{aligned}\Delta z &\approx \frac{\partial z}{\partial x_1}\frac{dx_1}{dt}\Delta t + \frac{\partial z}{\partial x_2}\frac{dx_2}{dt}\Delta t + \cdots + \frac{\partial z}{\partial x_n}\frac{dx_n}{dt}\Delta t \\ &= \left(\frac{\partial z}{\partial x_1}\frac{dx_1}{dt} + \cdots + \frac{\partial z}{\partial x_n}\frac{dx_n}{dt}\right)\Delta t.\end{aligned}$$

즉, t의 작은 변화가 여러 node x_i에 전달되고, 각 node에서 다시 z로 전달되는 모든 변화의 합이 최종적인 z의 변화가 된다.

한편 직접 z를 t의 함수로 생각하면

$$\Delta z \approx \frac{dz}{dt}\Delta t$$

이므로 위의 식과 그 모양을 비교하면 다음을 얻을수 있다.

$$\boxed{\frac{dz}{dt} = \frac{\partial z}{\partial x_1}\frac{dx_1}{dt} + \frac{\partial z}{\partial x_2}\frac{dx_2}{dt} + \cdots + \frac{\partial z}{\partial x_n}\frac{dx_n}{dt}}$$

각 항을 graph의 하나의 경로(path)로 생각하면

$$t \longrightarrow x_i \longrightarrow z$$

라는 경로를 따라 변화율이 곱해져

$$\frac{\partial z}{\partial x_i}\frac{dx_i}{dt}$$

가 된다.

그리고 t에서 z로 가는 경로가 여러 개 있으므로 각 경로를 통해 전달되는 변화율을 모두 더한다.

2.2 여러 입력변수를 가지는 경우

이제 중간변수 $x_1, \ldots, x_n$이 하나의 변수 t가 아니라 여러 변수

$$t_1, t_2, \ldots, t_k$$

에 의존한다고 하자.

즉,

$$x_i = x_i(t_1, t_2, \ldots, t_k), \qquad i = 1, \ldots, n$$

이고,

$$z = f(x_1, x_2, \ldots, x_n)$$

라고 하자.

이 관계를 graph로 나타내면 다음과 같다.

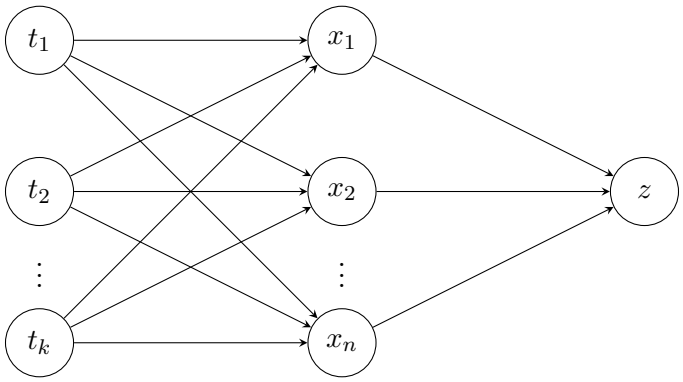


Figure 2: 여러 입력변수가 여러 중간 node를 거쳐 하나의 출력에 영향을 주는 구조

이제 특정 입력 t_j가 변할 때 출력 z가 얼마나 변하는지를 생각해 보자.

t_j 에서 z 까지 가는 경로는

$$t_j \to x_1 \to z, \qquad t_j \to x_2 \to z, \qquad \dots, \qquad t_j \to x_n \to z$$

와 같이 여러 개가 있다.

따라서

$$\boxed{\frac{\partial z}{\partial t_j} = \frac{\partial z}{\partial x_1}\frac{\partial x_1}{\partial t_j} + \frac{\partial z}{\partial x_2}\frac{\partial x_2}{\partial t_j} + \cdots + \frac{\partial z}{\partial x_n}\frac{\partial x_n}{\partial t_j}}$$

이다.

또는

$$\boxed{\frac{\partial z}{\partial t_j} = \sum_{i=1}^{n} \frac{\partial z}{\partial x_i}\frac{\partial x_i}{\partial t_j}}$$

로 쓸 수 있다.

Gradient와 Jacobian을 이용한 표현

앞에서 다변수 chain rule을

$$\boxed{\frac{\partial z}{\partial t_j} = \sum_{i=1}^{n} \frac{\partial z}{\partial x_i}\frac{\partial x_i}{\partial t_j}} \qquad (j = 1, \dots, k)$$

로 나타내었다.

이제 이 관계를 vector와 matrix를 이용하여 보다 간결하게 표현해 보자.

다음과 같이

$$\mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ \vdots \\ t_k \end{bmatrix} \in \mathbb{R}^k, \qquad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n$$

라고 하고,

$$\mathbf{x} = \mathbf{x}(\mathbf{t}), \qquad z = z(\mathbf{x})$$

라고 하자.

즉,

$$\mathbf{t} \longrightarrow \mathbf{x} \longrightarrow z$$

와 같은 함수의 합성을 생각하고 있다.

Gradient. 스칼라값 함수 $z = z(\mathbf{x})$의 $\mathbf{x}$에 대한 gradient를 다음과 같이 column vector로 정의한다.

$$\boxed{\nabla_{\mathbf{x}} z = \begin{bmatrix} \frac{\partial z}{\partial x_1} \\ \frac{\partial z}{\partial x_2} \\ \vdots \\ \frac{\partial z}{\partial x_n} \end{bmatrix} \in \mathbb{R}^n}$$

마찬가지로 z를 $\mathbf{t}$의 함수로 생각하면

$$\boxed{\nabla_{\mathbf{t}} z = \begin{bmatrix} \frac{\partial z}{\partial t_1} \\ \frac{\partial z}{\partial t_2} \\ \vdots \\ \frac{\partial z}{\partial t_k} \end{bmatrix} \in \mathbb{R}^k}$$

이다.

Jacobian. 벡터값 함수

$$\mathbf{x} = \mathbf{x}(\mathbf{t}) = \begin{bmatrix} x_1(\mathbf{t}) \\ x_2(\mathbf{t}) \\ \vdots \\ x_n(\mathbf{t}) \end{bmatrix}$$

의 Jacobian matrix를

$$\boxed{J_{\mathbf{x}}(\mathbf{t}) = \frac{\partial \mathbf{x}}{\partial \mathbf{t}} = \frac{\partial (x_1, ...x_n)}{\partial (t_1, ...t_k)} = \begin{bmatrix} \frac{\partial x_1}{\partial t_1} & \frac{\partial x_1}{\partial t_2} & \cdots & \frac{\partial x_1}{\partial t_k} \\ \frac{\partial x_2}{\partial t_1} & \frac{\partial x_2}{\partial t_2} & \cdots & \frac{\partial x_2}{\partial t_k} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial x_n}{\partial t_1} & \frac{\partial x_n}{\partial t_2} & \cdots & \frac{\partial x_n}{\partial t_k} \end{bmatrix} = \begin{bmatrix} (\nabla_{\mathbf{t}} x_1)^T \\ (\nabla_{\mathbf{t}} x_2)^T \\ \vdots \\ (\nabla_{\mathbf{t}} x_n)^T \end{bmatrix} \in \mathbb{R}^{n \times k}.}$$

로 정의한다. 따라서 그 transpose는

$$J_{\mathbf{x}}(\mathbf{t})^T = \begin{bmatrix} \frac{\partial x_1}{\partial t_1} & \frac{\partial x_2}{\partial t_1} & \cdots & \frac{\partial x_n}{\partial t_1} \\ \frac{\partial x_1}{\partial t_2} & \frac{\partial x_2}{\partial t_2} & \cdots & \frac{\partial x_n}{\partial t_2} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial x_1}{\partial t_k} & \frac{\partial x_2}{\partial t_k} & \cdots & \frac{\partial x_n}{\partial t_k} \end{bmatrix}$$

$$= \begin{bmatrix} \nabla_{\mathbf{t}} x_1 & \nabla_{\mathbf{t}} x_2 & \cdots & \nabla_{\mathbf{t}} x_n \end{bmatrix}.$$

즉, 변수 x_i 들의 벡터 $\mathbf{t}$에 대한 gradients를 열벡터로 갖는 행렬이다. 여기에 $\nabla_{\mathbf{x}} z$를 곱하면

$$J_{\mathbf{x}}(\mathbf{t})^T \nabla_{\mathbf{x}} z = \begin{bmatrix} \sum_{i=1}^{n} \frac{\partial x_i}{\partial t_1} \frac{\partial z}{\partial x_i} \\ \sum_{i=1}^{n} \frac{\partial x_i}{\partial t_2} \frac{\partial z}{\partial x_i} \\ \vdots \\ \sum_{i=1}^{n} \frac{\partial x_i}{\partial t_k} \frac{\partial z}{\partial x_i} \end{bmatrix} = \begin{bmatrix} \frac{\partial z}{\partial t_1} \\ \frac{\partial z}{\partial t_2} \\ \vdots \\ \frac{\partial z}{\partial t_k} \end{bmatrix} = \nabla_{\mathbf{t}} z = \frac{\partial z}{\partial \mathbf{t}}^T.$$

위 식의 양변에 transpose를 취하면 Jacobian을 이용한 chain rule을 다음과 같이 compact하게 쓸 수 있다.

$$\frac{\partial z}{\partial \mathbf{t}} = \nabla_{\mathbf{x}} z^T J_{\mathbf{x}}(\mathbf{t}) = \frac{\partial z}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \mathbf{t}}$$

한편, gradient를 이용해서 표시하면,

$$\nabla_{\mathbf{t}} z = \frac{\partial z}{\partial \mathbf{t}}^T = J_{\mathbf{x}}(\mathbf{t})^T \nabla_{\mathbf{x}} z = \frac{\partial \mathbf{x}}{\partial \mathbf{t}}^T \nabla_{\mathbf{x}} z$$

작은 변화량에 대한 표현. 이제 작은 변화량

$$\Delta \mathbf{t} = \begin{bmatrix} \Delta t_1 \\ \vdots \\ \Delta t_k \end{bmatrix}$$

을 생각하자.

스칼라함수 $z = z(\mathbf{t})$의 일차근사에 의해

$$\boxed{\Delta z \approx (\nabla_{\mathbf{t}} z)^T \Delta \mathbf{t} = \frac{\partial z}{\partial \mathbf{t}} \Delta \mathbf{t} = \frac{\partial z}{\partial x} \frac{\partial \mathbf{x}}{\partial \mathbf{t}} \Delta \mathbf{t}}$$

이다. 즉 입력층의 작은 변화량 $\Delta \mathbf{t}$가 출력층의 작은 변화량 Δz로 전달되는 과정에서 Jacobian을 타고 근사된다. 반면 출력 z의 $\mathbf{x}$에 대한 gradient $\nabla_{\mathbf{x}} z$는 Jacobian의 transpose를 타고 입력층으로 $\nabla_{\mathbf{t}} z$로 전달된다.

Jacobian과 gradient의 방향 스칼라값 함수 $z = z(\mathbf{x})$에 대해서 Jacobian은

$$\frac{\partial z}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial z}{\partial x_1} & \cdots & \frac{\partial z}{\partial x_n} \end{bmatrix}$$

와 같은 row vector이고, gradient는

$$\nabla_{\mathbf{x}} z = \begin{bmatrix} \frac{\partial z}{\partial x_1} \\ \vdots \\ \frac{\partial z}{\partial x_n} \end{bmatrix}$$

와 같은 column vector이다. 따라서

$$\boxed{\frac{\partial z}{\partial \mathbf{x}} = (\nabla_{\mathbf{x}} z)^T}$$

의 관계가 있다.

한편 벡터값 함수 $\mathbf{x} = \mathbf{x}(\mathbf{t})$에 대해서는

$$\Delta \mathbf{x} \approx \frac{\partial \mathbf{x}}{\partial \mathbf{t}} \Delta \mathbf{t}$$

이므로 작은 변화량은 Jacobian을 곱하면서 함수의 진행 방향으로 전달된다.

반대로 z가 scalar일 때 gradient는

$$\boxed{\nabla_{\mathbf{t}} z = \left(\frac{\partial \mathbf{x}}{\partial \mathbf{t}} \right)^T \nabla_{\mathbf{x}} z}$$

이므로 뒤쪽 node에서 계산된 gradient는 Jacobian의 transpose를 곱하면서 앞쪽 node로 전달된다.

즉,

$$\boxed{\Delta \mathbf{t} \xrightarrow{J} \Delta \mathbf{x}, \qquad \nabla_{\mathbf{x}} z \xrightarrow{J^T} \nabla_{\mathbf{t}} z}$$

이다. 이러한 "Jacobian은 forward 방향, Jacobian transpose는 gradient의 backward 방향"이라는 구조가 이후 backpropagation의 기본 원리가 된다.

예제

예 1.

$$z = z(x, y) = 3x + 5y + 2, \qquad x = x(t, s) = 2t + s + 1, \qquad y = y(t, s) = 3t - 2s - 1$$

이라고 하자.

(1) 각 편도함수를 직접 계산하는 방법

$$z_t = z_x x_t + z_y y_t = 3 \cdot 2 + 5 \cdot 3 = 21,$$

$$z_s = z_x x_s + z_y y_s = 3 \cdot 1 + 5(-2) = -7.$$

따라서

$$\boxed{\nabla_{(t,s)} z = \begin{bmatrix} z_t \\ z_s \end{bmatrix} = \begin{bmatrix} 21 \\ -7 \end{bmatrix}}$$

이다.

한편 x, y를 z에 직접 대입하면

$$z = 3(2t + s + 1) + 5(3t - 2s - 1) + 2 = 21t - 7s$$

이므로

$$z_t = 21, \qquad z_s = -7$$

이고, 위의 결과와 일치한다.

(2) Jacobian과 gradient를 이용한 방법

$$\frac{\partial(x, y)}{\partial(t, s)} = \begin{bmatrix} x_t & x_s \\ y_t & y_s \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 3 & -2 \end{bmatrix},$$

그리고

$$\nabla_{(x,y)} z = \begin{bmatrix} z_x \\ z_y \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}.$$

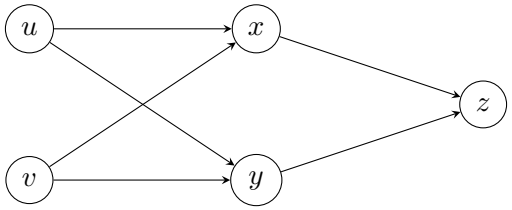
따라서

$$\boxed{\nabla_{(t,s)} z = \left(\frac{\partial(x, y)}{\partial(t, s)} \right)^T \nabla_{(x,y)} z = \begin{bmatrix} 2 & 3 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} 3 \\ 5 \end{bmatrix} = \begin{bmatrix} 21 \\ -7 \end{bmatrix}}$$

예 2.

$$z = z(x, y) = e^{2x} + \ln y, \qquad x = x(u, v) = u + 2v, \qquad y = y(u, v) = u^2 v$$

에 대하여 z의 u, v에 대한 편도함수를 구해보자.



(1) 각 편도함수를 직접 계산하는 방법 다변수 chain rule에 의해

$$z_u = z_x x_u + z_y y_u, \qquad z_v = z_x x_v + z_y y_v.$$

각 편도함수는

$$z_x = 2e^{2x}, \qquad z_y = \frac{1}{y}, \qquad x_u = 1, \qquad x_v = 2, \qquad y_u = 2uv, \qquad y_v = u^2$$

이므로

$$z_u = z_x x_u + z_y y_u = 2e^{2x} \cdot 1 + \frac{1}{y}(2uv) = 2e^{2u+4v} + \frac{2}{u},$$

$$z_v = z_x x_v + z_y y_v = 2e^{2x} \cdot 2 + \frac{1}{y}u^2 = 4e^{2u+4v} + \frac{1}{v}.$$

따라서

$$\boxed{\nabla_{(u,v)} z = \begin{bmatrix} z_u \\ z_v \end{bmatrix} = \begin{bmatrix} 2e^{2u+4v} + \dfrac{2}{u} \\ 4e^{2u+4v} + \dfrac{1}{v} \end{bmatrix}}$$

이다.

(2) Jacobian과 gradient를 이용한 방법 중간변수 (x, y)의 입력변수 (u, v)에 대한 Jacobian은

$$\frac{\partial(x, y)}{\partial(u, v)} = \begin{bmatrix} x_u & x_v \\ y_u & y_v \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2uv & u^2 \end{bmatrix}$$

이고,

$$\nabla_{(x,y)} z = \begin{bmatrix} z_x \\ z_y \end{bmatrix} = \begin{bmatrix} 2e^{2x} \\ \dfrac{1}{y} \end{bmatrix}.$$

따라서

$$\nabla_{(u,v)} z = \left(\frac{\partial(x,y)}{\partial(u,v)}\right)^T \nabla_{(x,y)} z = \begin{bmatrix} 1 & 2uv \\ 2 & u^2 \end{bmatrix} \begin{bmatrix} 2e^{2x} \\ \frac{1}{y} \end{bmatrix}$$

$$= \begin{bmatrix} 2e^{2x} + \frac{2uv}{y} \\ 4e^{2x} + \frac{u^2}{y} \end{bmatrix} = \begin{bmatrix} 2e^{2u+4v} + \frac{2}{u} \\ 4e^{2u+4v} + \frac{1}{v} \end{bmatrix}.$$

즉, z_u와 z_v를 각각 계산하는 두 개의 chain rule을 Jacobian과 gradient의 하나의 matrix-vector multiplication으로 표현할 수 있다.

Gradient descent의 한 step 이제 z를 (u,v)에 대한 목적함수로 생각하고, gradient descent를 한 step 수행해보자. 초기값을

$$u_0 = -2, \qquad v_0 = 1$$

로 두면

$$x_0 = u_0 + 2v_0 = 0, \qquad y_0 = u_0^2 v_0 = 4,$$

따라서

$$z_0 = e^{2x_0} + \ln y_0 = 1 + \ln 4 \approx 2.3863.$$

이제 (u,v)에 대한 z의 gradient를 직접 미분하여 구하지 않고, computational graph

$$(u,v) \longrightarrow (x,y) \longrightarrow z$$

를 따라 뒤에서부터 차례로 계산해보자.

먼저 마지막 단계

$$(x,y) \longrightarrow z$$

에서의 gradient는

$$\nabla_{(x,y)} z = \begin{bmatrix} z_x \\ z_y \end{bmatrix} = \begin{bmatrix} 2e^{2x} \\ \frac{1}{y} \end{bmatrix}.$$

$(x_0, y_0) = (0,4)$에서

$$\nabla_{(x,y)} z = \begin{bmatrix} 2 \\ \frac{1}{4} \end{bmatrix}.$$

다음으로 앞 단계

$$(u,v) \longrightarrow (x,y)$$

의 Jacobian은

$$\frac{\partial(x,y)}{\partial(u,v)} = \begin{bmatrix} x_u & x_v \\ y_u & y_v \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2uv & u^2 \end{bmatrix}.$$

$(u_0, v_0) = (-2, 1)$ 에서

$$\frac{\partial(x,y)}{\partial(u,v)} = \begin{bmatrix} 1 & 2 \\ -4 & 4 \end{bmatrix}.$$

따라서 chain rule의 matrix form을 이용하면

$$\begin{aligned} \nabla_{(u,v)} z &= \left(\frac{\partial(x,y)}{\partial(u,v)}\right)^T \nabla_{(x,y)} z \\ &= \begin{bmatrix} 1 & -4 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ \frac{1}{4} \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \end{bmatrix}. \end{aligned}$$

즉,

$$\boxed{\nabla_{(x,y)} z \quad \xrightarrow{\left(\frac{\partial(x,y)}{\partial(u,v)}\right)^T} \quad \nabla_{(u,v)} z}$$

와 같이 뒤쪽 node에서 구한 gradient가 Jacobian transpose와의 matrix-vector multiplication을 통하여 앞쪽 node로 전달된다.

이제 learning rate를 $\eta = 0.01$ 로 두면

$$\begin{aligned} \begin{bmatrix} u_1 \\ v_1 \end{bmatrix} &= \begin{bmatrix} u_0 \\ v_0 \end{bmatrix} - \eta \nabla_{(u,v)} z \\ &= \begin{bmatrix} -2 \\ 1 \end{bmatrix} - 0.01 \begin{bmatrix} 1 \\ 5 \end{bmatrix} = \begin{bmatrix} -2.01 \\ 0.95 \end{bmatrix}. \end{aligned}$$

새로운 값에서

$$x_1 = u_1 + 2v_1 = -0.11, \qquad y_1 = u_1^2 v_1 \approx 3.8381,$$

따라서

$$z_1 = e^{2x_1} + \ln y_1 \approx 2.1475.$$

결과적으로

$$\boxed{z_0 \approx 2.3863 \quad \longrightarrow \quad z_1 \approx 2.1475}$$

로 감소한다.

2.3 입력과 출력이 모두 여러 개인 경우

이제 출력도 하나가 아니라 여러 개라고 하자.

$$\mathbf{t} = (t_1, \ldots, t_k), \qquad \mathbf{x} = (x_1, \ldots, x_n), \qquad \mathbf{z} = (z_1, \ldots, z_\ell)$$

이고

$$\mathbf{t} \longrightarrow \mathbf{x} \longrightarrow \mathbf{z}$$

와 같은 계산구조를 생각하자.

각 출력 z_r는 모든 중간변수 x_i의 함수이고, 각 x_i는 다시 모든 입력변수 t_j의 함수이다.

그러면 각 pair (z_r, t_j)에 대하여

$$\boxed{\frac{\partial z_r}{\partial t_j} = \sum_{i=1}^{n} \frac{\partial z_r}{\partial x_i} \frac{\partial x_i}{\partial t_j}}$$

가 성립한다.

이 관계를 graph로 보면 다음과 같다.

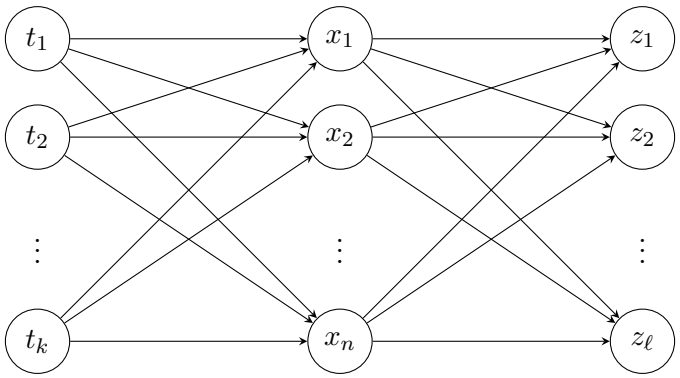


Figure 3: 입력과 출력이 모두 여러 개인 vector-valued multivariable function

따라서 벡터값 함수에서도 본질은 동일하다. 각 출력 node z_r에 대해 하나씩 생각하면 된다.

예를 들어 z_1의 t_2에 대한 변화율을 알고 싶다면 t_2에서 z_1까지 연결되는 모든 경로를 찾고,

$$t_2 \to x_i \to z_1$$

각 경로의 derivative들을 곱한 뒤, 모든 경로의 결과를 합하면 된다.

Jacobian을 이용한 합성함수 미분법 이제 위의 관계를 Jacobian matrix를 이용하여 한꺼번에 표현해보자. 다음의 합성함수를 생각하자.

$$\mathbf{t} = (t_1, \ldots, t_k) \longrightarrow \mathbf{x} = (x_1, \ldots, x_n) \longrightarrow \mathbf{z} = (z_1, \ldots, z_\ell).$$

$\mathbf{z}$의 $\mathbf{t}$에 대한 Jacobian은

$$\frac{\partial(z_1, \ldots, z_\ell)}{\partial(t_1, \ldots, t_k)} = \begin{bmatrix} \dfrac{\partial z_1}{\partial t_1} & \cdots & \dfrac{\partial z_1}{\partial t_k} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial z_\ell}{\partial t_1} & \cdots & \dfrac{\partial z_\ell}{\partial t_k} \end{bmatrix}.$$

한편 중간변수 $\mathbf{x}$를 기준으로 두 Jacobian을 각각 쓰면

$$\frac{\partial(z_1,\ldots,z_\ell)}{\partial(x_1,\ldots,x_n)} = \begin{bmatrix} \frac{\partial z_1}{\partial x_1} & \cdots & \frac{\partial z_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial z_\ell}{\partial x_1} & \cdots & \frac{\partial z_\ell}{\partial x_n} \end{bmatrix}$$

이고,

$$\frac{\partial(x_1,\ldots,x_n)}{\partial(t_1,\ldots,t_k)} = \begin{bmatrix} \frac{\partial x_1}{\partial t_1} & \cdots & \frac{\partial x_1}{\partial t_k} \\ \vdots & \ddots & \vdots \\ \frac{\partial x_n}{\partial t_1} & \cdots & \frac{\partial x_n}{\partial t_k} \end{bmatrix}.$$

이 두 Jacobian을 곱한 행렬의 (r,j) 번째 원소는

$$\sum_{i=1}^{n} \frac{\partial z_r}{\partial x_i}\frac{\partial x_i}{\partial t_j} = \frac{\partial z_r}{\partial t_j}.$$

따라서

$$\boxed{\frac{\partial(z_1,\ldots,z_\ell)}{\partial(t_1,\ldots,t_k)} = \frac{\partial(z_1,\ldots,z_\ell)}{\partial(x_1,\ldots,x_n)}\frac{\partial(x_1,\ldots,x_n)}{\partial(t_1,\ldots,t_k)}}$$

를 얻는다.

즉, 차원으로 보면

$$(\ell \times k) = (\ell \times n)(n \times k)$$

이고, 다변수함수의 chain rule은 Jacobian matrix의 곱으로 compact하게 표현된다.

또한 작은 변화량을 생각하면

$$\Delta\mathbf{x} \approx \frac{\partial \mathbf{x}}{\partial \mathbf{t}}\Delta\mathbf{t}, \qquad \Delta\mathbf{z} \approx \frac{\partial \mathbf{z}}{\partial \mathbf{x}}\Delta\mathbf{x},$$

이므로

$$\boxed{\Delta\mathbf{z} \approx \frac{\partial \mathbf{z}}{\partial \mathbf{x}}\frac{\partial \mathbf{x}}{\partial \mathbf{t}}\Delta\mathbf{t} = \frac{\partial \mathbf{z}}{\partial \mathbf{t}}\Delta\mathbf{t}}$$

가 된다.

2.4 여러 층으로 이루어진 함수의 합성

이제 중간 node가 한 층이 아니라 여러 층 존재한다고 하자.

예를 들어

$$\mathbf{t} \longrightarrow \mathbf{x} \longrightarrow \mathbf{y} \longrightarrow \mathbf{z}$$

와 같은 계산구조를 생각할 수 있다.

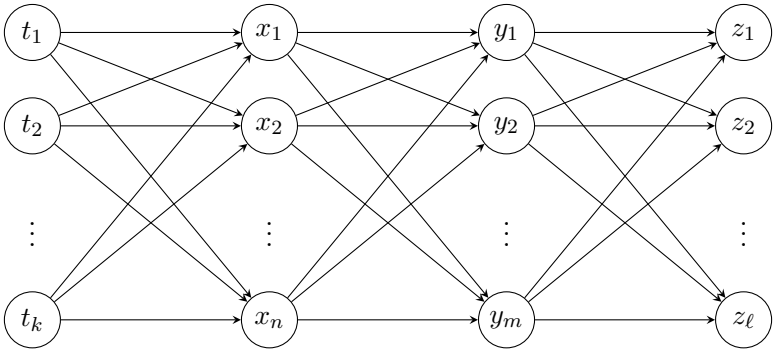


Figure 4: 여러 층의 node로 이루어진 함수의 반복적인 합성

예를 들어 z_r의 t_j에 대한 미분을 생각하면, t_j에서 z_r까지 가는 경로는 매우 많아진다.

하나의 경로

$$t_j \longrightarrow x_i \longrightarrow y_q \longrightarrow z_r$$

를 따라 전달되는 변화율은

$$\frac{\partial z_r}{\partial y_q}\frac{\partial y_q}{\partial x_i}\frac{\partial x_i}{\partial t_j}$$

이고, 가능한 모든 i, q를 통하는 경로를 합하면

$$\boxed{\frac{\partial z_r}{\partial t_j} = \sum_q \sum_i \frac{\partial z_r}{\partial y_q}\frac{\partial y_q}{\partial x_i}\frac{\partial x_i}{\partial t_j}}$$

를 얻는다.

따라서 층의 수가 증가하더라도 기본 원리는 변하지 않는다.

예

다음과 같은 합성함수를 생각하자.

$$(t_1, t_2) \longrightarrow (x_1, x_2) \longrightarrow (y_1, y_2) \longrightarrow z,$$

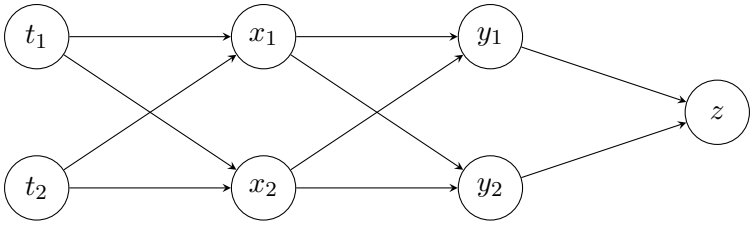
여기서

$$x_1 = t_1 + t_2, \qquad x_2 = t_1 - t_2,$$

$$y_1 = x_1^2 + x_2, \qquad y_2 = x_1 + 2x_2,$$

$$z = y_1^2 + y_2$$

라고 하자.



각 층의 Jacobian은

$$\frac{\partial(x_1, x_2)}{\partial(t_1, t_2)} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix},$$

$$\frac{\partial(y_1, y_2)}{\partial(x_1, x_2)} = \begin{bmatrix} 2x_1 & 1 \\ 1 & 2 \end{bmatrix},$$

그리고 마지막 출력 z의 (y_1, y_2)에 대한 Jacobian은

$$\frac{\partial z}{\partial(y_1, y_2)} = \begin{bmatrix} 2y_1 & 1 \end{bmatrix}.$$

따라서 Jacobian의 chain rule에 의해

$$\boxed{\frac{\partial z}{\partial(t_1, t_2)} = \frac{\partial z}{\partial(y_1, y_2)} \frac{\partial(y_1, y_2)}{\partial(x_1, x_2)} \frac{\partial(x_1, x_2)}{\partial(t_1, t_2)}}$$

이다.

차원을 확인하면

$$(1 \times 2)(2 \times 2)(2 \times 2) = (1 \times 2)$$

로 정확히 맞는다.

한편 gradient는 scalar 함수의 Jacobian을 transpose한 것이므로,

$$\nabla_{(t_1,t_2)} z = \left(\frac{\partial z}{\partial (t_1, t_2)} \right)^T.$$

앞의 Jacobian chain rule을 transpose하면

$$\begin{aligned}
\nabla_{(t_1,t_2)} z &= \left[\frac{\partial z}{\partial (y_1, y_2)} \frac{\partial (y_1, y_2)}{\partial (x_1, x_2)} \frac{\partial (x_1, x_2)}{\partial (t_1, t_2)} \right]^T \\
&= \left(\frac{\partial (x_1, x_2)}{\partial (t_1, t_2)} \right)^T \left(\frac{\partial (y_1, y_2)}{\partial (x_1, x_2)} \right)^T \left(\frac{\partial z}{\partial (y_1, y_2)} \right)^T \\
&= \left(\frac{\partial (x_1, x_2)}{\partial (t_1, t_2)} \right)^T \left(\frac{\partial (y_1, y_2)}{\partial (x_1, x_2)} \right)^T \nabla_{(y_1,y_2)} z.
\end{aligned}$$

즉 gradient는 출력에서 시작하여

$$\boxed{\nabla_{(y_1,y_2)} z \xrightarrow{\left(\frac{\partial (y_1,y_2)}{\partial (x_1,x_2)}\right)^T} \nabla_{(x_1,x_2)} z \xrightarrow{\left(\frac{\partial (x_1,x_2)}{\partial (t_1,t_2)}\right)^T} \nabla_{(t_1,t_2)} z}$$

와 같이 각 층의 Jacobian transpose를 곱하면서 graph의 반대 방향으로 전달된다.

3 Linear Regression을 Computational Graph로 다시 보기

이제 이미 알고 있는 선형회귀를 chain rule의 관점에서 다시 해석한다. 데이터

$$(x_i, y_i), \qquad i = 1, \ldots, N$$

가 주어졌다고 하자.

모델은

$$\widehat{y}_i = w x_i + b$$

이고, mean squared error는

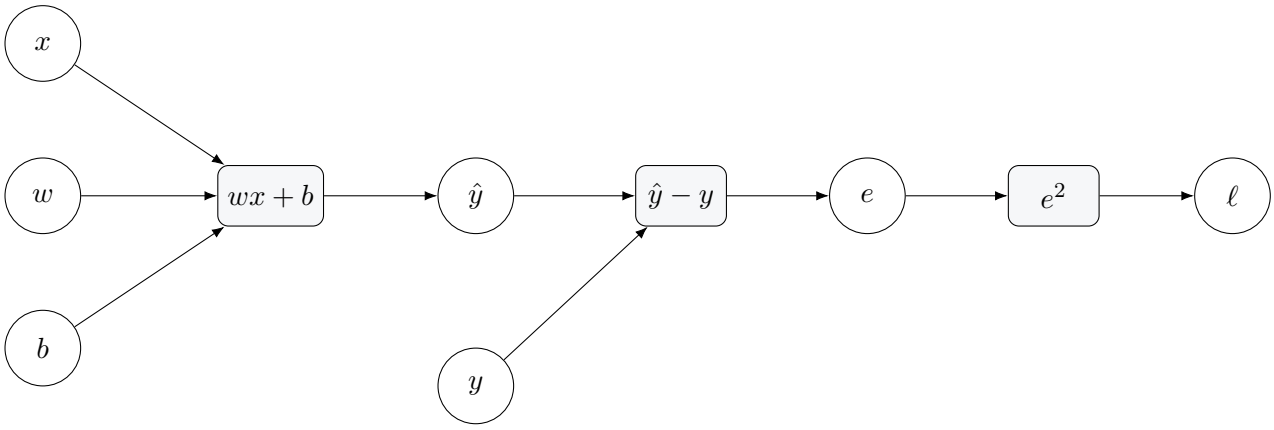
$$\boxed{L(w, b) = \frac{1}{N} \sum_{i=1}^{N} (\widehat{y}_i - y_i)^2}$$

이다.

3.1 한 개의 데이터에 대한 계산 그래프

먼저 하나의 datum (x, y) 만 생각하자.

$$\widehat{y} = wx + b, \qquad e = \widehat{y} - y, \qquad \ell = e^2.$$



Forward 계산은 단순히

$$(w, b, x) \longrightarrow \widehat{y} \longrightarrow e \longrightarrow \ell$$

순서로 진행한다.

3.2 Local derivatives

각 edge에 해당하는 local derivative를 계산하면

$$\frac{\partial \ell}{\partial e} = 2e, \qquad \frac{\partial e}{\partial \widehat{y}} = 1,$$

그리고

$$\frac{\partial \widehat{y}}{\partial w} = x, \qquad \frac{\partial \widehat{y}}{\partial b} = 1.$$

따라서 chain rule에 의해

$$\boxed{\frac{\partial \ell}{\partial w} = \frac{\partial \ell}{\partial e}\frac{\partial e}{\partial \widehat{y}}\frac{\partial \widehat{y}}{\partial w} = 2ex}$$

이고

$$\boxed{\frac{\partial \ell}{\partial b} = \frac{\partial \ell}{\partial e}\frac{\partial e}{\partial \widehat{y}}\frac{\partial \widehat{y}}{\partial b} = 2e}$$

이다.

즉, loss에서 시작하여 computational graph를 반대 방향으로 따라가면서 각 edge의 local derivative를 곱한다.

3.3 전체 데이터에 대한 gradient

$$L(w, b) = \frac{1}{N}\sum_{i=1}^{N} \ell_i$$

이므로

$$\frac{\partial L}{\partial w} = \frac{1}{N}\sum_{i=1}^{N}\frac{\partial \ell_i}{\partial w} = \boxed{\frac{2}{N}\sum_{i=1}^{N}(\widehat{y}_i - y_i)x_i}$$

이고

$$\frac{\partial L}{\partial b} = \boxed{\frac{2}{N}\sum_{i=1}^{N}(\widehat{y}_i - y_i)}$$

이다.

Gradient vector는

$$\nabla L(w,b) = \begin{bmatrix} \partial L/\partial w \\ \partial L/\partial b \end{bmatrix}.$$

Gradient descent는

$$\boxed{\begin{bmatrix} w_{k+1} \\ b_{k+1} \end{bmatrix} = \begin{bmatrix} w_k \\ b_k \end{bmatrix} - \eta \begin{bmatrix} \partial L/\partial w \\ \partial L/\partial b \end{bmatrix}}$$

이다.

여기에서 구분해야 할 두 가지

Backpropagation은 gradient를 계산하는 방법이다.
Gradient descent는 계산된 gradient를 사용하여 parameter를 업데이트하는 방법이다.
즉,

$$\boxed{\text{Forward} \rightarrow \text{Loss} \rightarrow \text{Backward} \rightarrow \text{Gradient} \rightarrow \text{Parameter Update}}$$

이다.

4 숫자로 Gradient의 흐름을 끝까지 추적해 보기

다음 세 개의 데이터를 생각하자.

$$(1,1), \qquad (2,2), \qquad (3,2).$$

초기값을

$$w_0 = 1, \qquad b_0 = 1$$

로 둔다. 그러면 $\widehat{y} = x + 1$이다.

x_i	y_i	$\widehat{y}_i$	e_i	$\ell_i = e_i^2$	$\partial \ell_i/\partial w$	$\partial \ell_i/\partial b$
1	1	2	1	1	$2(1)(1) = 2$	$2(1) = 2$
2	2	3	1	1	$2(1)(2) = 4$	$2(1) = 2$
3	2	4	2	4	$2(2)(3) = 12$	$2(2) = 4$

따라서

$$\frac{\partial L}{\partial w} = \frac{2+4+12}{3} = 6, \qquad \frac{\partial L}{\partial b} = \frac{2+2+4}{3} = \frac{8}{3}.$$

학습률을 $\eta = 0.03$이라 하면 한 번의 gradient descent step은

$$w_1 = 1 - 0.03(6) = 0.82,$$

$$b_1 = 1 - 0.03\left(\frac{8}{3}\right) = 0.92.$$

5 3차시 실습: NumPy Hard Coding

이번 실습에서는 PyTorch의 autograd를 사용하지 않는다. 수학적으로 유도한 모든 값을 직접 계산한다.

5.1 실습 목표

코드 안에서 다음 다섯 단계를 눈으로 확인한다.

$$\boxed{\text{data} \rightarrow \text{forward} \rightarrow \text{loss} \rightarrow \text{backward} \rightarrow \text{parameter update}}$$

5.2 Step 1. 데이터와 모델

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1., 2., 3., 4., 5.])
y = np.array([2.2, 2.8, 3.6, 4.5, 5.1])

w = 0.0
b = 0.0
learning_rate = 0.01
epochs = 200
N = len(x)
```

모델은

$$\widehat{y} = wx + b$$

이다.

5.3 Step 2. Forward propagation

```
y_hat = w * x + b
error = y_hat - y
loss = np.mean(error**2)
```

이 세 줄은 수학적으로

$$\widehat{y}_i = wx_i + b, \qquad e_i = \widehat{y}_i - y_i, \qquad L = \frac{1}{N}\sum_i e_i^2$$

를 계산한다.

5.4 Step 3. Backward propagation

이번에는 최종 공식을 바로 쓰지 말고 local derivative를 코드에 그대로 남긴다.

```
dl_de = 2.0 * error

de_dyhat = 1.0

dyhat_dw = x
dyhat_db = 1.0

# Chain rule for each sample
sample_grad_w = dl_de * de_dyhat * dyhat_dw
sample_grad_b = dl_de * de_dyhat * dyhat_db

# Mean over all samples
dL_dw = np.mean(sample_grad_w)
dL_db = np.mean(sample_grad_b)
```

수학식과 정확히 대응시키면

$$\frac{\partial \ell_i}{\partial w} = \underbrace{\frac{\partial \ell_i}{\partial e_i}}_{2e_i} \underbrace{\frac{\partial e_i}{\partial \widehat{y}_i}}_{1} \underbrace{\frac{\partial \widehat{y}_i}{\partial w}}_{x_i}$$

이고,

$$\frac{\partial \ell_i}{\partial b} = \underbrace{\frac{\partial \ell_i}{\partial e_i}}_{2e_i} \underbrace{\frac{\partial e_i}{\partial \widehat{y}_i}}_{1} \underbrace{\frac{\partial \widehat{y}_i}{\partial b}}_{1}.$$

5.5 Step 4. Parameter update

```
w -= learning_rate * dL_dw
b -= learning_rate * dL_db
```

즉,

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \qquad b \leftarrow b - \eta \frac{\partial L}{\partial b}.$$

5.6 Step 5. 전체 training loop

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1., 2., 3., 4., 5.])
y = np.array([2.2, 2.8, 3.6, 4.5, 5.1])

w = 0.0
b = 0.0
learning_rate = 0.01
epochs = 200
N = len(x)
```

```
12
13 loss_history = []
14
15 for epoch in range(epochs):
16     # ---------------------------------
17     # 1. Forward
18     # ---------------------------------
19     y_hat = w * x + b
20     error = y_hat - y
21     loss = np.mean(error**2)
22
23     # ---------------------------------
24     # 2. Backward: local derivatives
25     # ---------------------------------
26     dl_de = 2.0 * error
27     de_dyhat = 1.0
28     dyhat_dw = x
29     dyhat_db = 1.0
30
31     sample_grad_w = dl_de * de_dyhat * dyhat_dw
32     sample_grad_b = dl_de * de_dyhat * dyhat_db
33
34     dL_dw = np.mean(sample_grad_w)
35     dL_db = np.mean(sample_grad_b)
36
37     # ---------------------------------
38     # 3. Gradient-descent step
39     # ---------------------------------
40     w -= learning_rate * dL_dw
41     b -= learning_rate * dL_db
42
43     loss_history.append(loss)
44
45     if epoch % 20 == 0:
46         print(
47             f"epoch={epoch:3d}, loss={loss:.6f}, "
48             f"w={w:.4f}, b={b:.4f}, "
49             f"dL_dw={dL_dw:.4f}, dL_db={dL_db:.4f}"
50         )
51
52 print("final w =", w)
53 print("final b =", b)
54
55 # Plot fitted line
56 x_plot = np.linspace(0.5, 5.5, 100)
57 y_plot = w * x_plot + b
58
59 plt.figure(figsize=(10, 4))
60
61 plt.subplot(1, 2, 1)
62 plt.scatter(x, y, label="data")
```

```
plt.plot(x_plot, y_plot, label="fitted line")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(loss_history)
plt.xlabel("epoch")
plt.ylabel("MSE loss")
plt.grid(True)

plt.tight_layout()
plt.show()
```

5.7 Step 6. Gradient check: 우리가 유도한 미분이 맞는가?

Backpropagation으로 계산한 gradient를 numerical differentiation과 비교해 볼 수 있다.

```
def compute_loss(x, y, w, b):
    y_hat = w * x + b
    return np.mean((y_hat - y)**2)


def numerical_gradient(x, y, w, b, h=1e-5):
    dL_dw = (
        compute_loss(x, y, w + h, b)
        - compute_loss(x, y, w - h, b)
    ) / (2*h)

    dL_db = (
        compute_loss(x, y, w, b + h)
        - compute_loss(x, y, w, b - h)
    ) / (2*h)

    return dL_dw, dL_db
```

예를 들어 같은 (w, b)에서

$$\left(\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}\right)_{\text{backprop}} \approx \left(\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}\right)_{\text{finite difference}}$$

인지 확인한다.

6 연습문제

1. 다음 함수의 도함수를 중간 변수를 명시하여 chain rule로 계산하라.

$$y = \log\left(1 + e^{2x}\right).$$

2.

$$z = x^2 y + \sin y, \qquad x = t^2, \qquad y = e^t$$

일 때 dz/dt를 구하라.

3. 다음 vector-valued function의 Jacobian을 구하라.

$$F(x_1, x_2) = \begin{bmatrix} x_1^2 + x_2 \\ \sin(x_1 x_2) \end{bmatrix}.$$

4. 선형회귀에서 loss를

$$L(w, b) = \frac{1}{2N} \sum_{i=1}^{N} (w x_i + b - y_i)^2$$

로 정의하면 $\partial L/\partial w$, $\partial L/\partial b$는 어떻게 바뀌는가? $1/2$를 붙이는 이유를 설명하라.

5. 데이터

$$(1, 2), \qquad (2, 2), \qquad (4, 5)$$

와 초기값 $w = 1/2$, $b = 1$이 주어졌다고 하자. 한 번의 forward/backward를 직접 계산하여 $\partial L/\partial w$, $\partial L/\partial b$를 구하고, $\eta = 0.05$로 한 번 update한 (w, b)를 계산하라.

6. 코드에서 `learning_rate`를 0.001, 0.01, 0.1, 0.5로 바꾸어 loss history를 비교하라. 학습률이 너무 작거나 너무 클 때 어떤 현상이 나타나는지 설명하라.

참고문헌

- M. P. Deisenroth, A. A. Faisal, C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, Chapter 5 (Vector Calculus), especially Sections 5.2, 5.3, and 5.6.